Supply Chain Analysis 📦🔍

1. Project Overview

This dataset acts as a Supply Chain Efficiency Monitor, providing a complete view of how products move from suppliers to customers while tracking time, inventory, and costs. It is structured around three key areas: logistics, which helps identify delays in processing and shipping; inventory, which ensures stock levels are balanced and highlights risks like shortages or overstocking; and financials, which evaluate overall profitability by analyzing sales and various costs. Together, these elements make the dataset a powerful diagnostic tool that helps identify inefficiencies, reduce costs, and improve overall business performance. The main goal is to understand how warehouses and product categories are performing by looking at inventory, demand, costs and risks. By cleaning and organizing the data, the project aims to convert raw data into meaningful insights.

2. Key Areas of Analysis

- **Inventory & Demand →** Compare available stock with demand forecast to spot shortages or excess.
- **Risk Metrics →** Track risks like stockouts, product returns, and damaged goods.
- **Cost Analysis →** Study operational, storage, and transport costs to control expenses.
- **Performance →** Measure monthly sales and net profit to check overall business health.
- **Customer Satisfaction →** Use ratings and return rates to understand customer experience.

3. Project Objectives

- **Clean the data** → Fix errors, remove duplicates, and prepare the dataset for analysis.
- **Study inventory vs demand** → Compare stock with demand forecast to spot shortages or surpluses.
- **Measure risks** → Track issues like stockouts, product returns, and damaged goods.

4. Dataset Summary

- The data has **500 rows** (warehouse records) and **20 columns** (details about each warehouse and products).
- It includes **warehouse details** like Warehouse ID, Location, and Employee Count.
- It tracks **inventory and demand**: Current Stock, Demand Forecast, Warehouse

Capacity, Backorder Quantity.

- It shows **risk metrics**: Stockout Risk, Damaged Goods, Return Rate, Customer Rating. It includes **financials**: Operational, Storage, Transportation Cost.

5. Exploratory Data Analysis using Python

- **Data Loading** → We opened the dataset in Python using the pandas library. This step makes the data ready to work with.

- **Initial Exploration** → We checked the basic structure of the dataset using df.info () to see number of rows, columns, and data types.

- **Summary Statistics** → We used df.describe() to quickly look at averages, minimums, maximums, and other statistics for the numeric columns.

```
# DataFrame Summary
df.info()
```

✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 20 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Warehouse_ID           500 non-null    str
 1   Location               500 non-null    str
 2   Current_Stock          500 non-null    int64
 3   Demand_Forecast        500 non-null    int64
 4   Lead_Time_Days         500 non-null    int64
 5   Shipping_Time_Days     500 non-null    int64
 6   Stockout_Risk          500 non-null    int64
 7   Operational_Cost       500 non-null    int64
 8   Supplier_ID            500 non-null    str
 9   Product_Category       500 non-null    str
 10  Monthly_Sales          500 non-null    int64
 11  Order_Processing_Time  500 non-null    float64
 12  Return_Rate            500 non-null    float64
 13  Customer_Rating        500 non-null    float64
 14  Warehouse_Capacity     500 non-null    int64
 15  Storage_Cost           500 non-null    int64
 16  Transportation_Cost    500 non-null    int64
 17  Backorder_Quantity     500 non-null    int64
```

```
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 20 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Warehouse_ID           500 non-null    str
 1   Location               500 non-null    str
 2   Current_Stock          500 non-null    int64
 3   Demand_Forecast        500 non-null    int64
 4   Lead_Time_Days         500 non-null    int64
 5   Shipping_Time_Days     500 non-null    int64
 6   Stockout_Risk          500 non-null    int64
 7   Operational_Cost       500 non-null    int64
 8   Supplier_ID            500 non-null    str
 9   Product_Category       500 non-null    str
 10  Monthly_Sales          500 non-null    int64
 11  Order_Processing_Time  500 non-null    float64
 12  Return_Rate            500 non-null    float64
 13  Customer_Rating        500 non-null    float64
 14  Warehouse_Capacity     500 non-null    int64
 15  Storage_Cost           500 non-null    int64
 16  Transportation_Cost    500 non-null    int64
 17  Backorder_Quantity     500 non-null    int64
 18  Damaged_Goods          500 non-null    int64
 19  Employee_Count         500 non-null    int64
dtypes: float64(3), int64(13), str(4)
memory usage: 78.3 KB
```

```python
# Describe Dataset
df.describe()
```

✓ 0.1s Python

	Current_Stock	Demand_Forecast	Lead_Time_Days	Shipping_Time_Days	Stockout_Risk	Operational_Cost	Monthly_Sales	Order_Processing_Time	Return_Rat
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.00000
mean	2722.822000	2957.050000	5.958000	3.948000	14.944000	65886.354000	5335.572000	2.750462	5.22936
std	1314.878786	1398.763988	2.630223	1.921635	5.923431	20283.576821	2619.353688	1.262967	2.66144
min	519.000000	402.000000	2.000000	1.000000	5.000000	30165.000000	1004.000000	0.505159	0.50675
25%	1565.250000	1745.750000	4.000000	2.000000	10.000000	48731.750000	2978.000000	1.644623	3.08548
50%	2670.000000	3094.000000	6.000000	4.000000	15.000000	65733.500000	5234.000000	2.817771	5.12813
75%	3842.750000	4152.500000	8.000000	6.000000	20.000000	82900.250000	7550.750000	3.782273	7.40640
max	4998.000000	5200.000000	10.000000	7.000000	25.000000	99965.000000	9931.000000	4.954139	9.99246

```python
# Describe Dataset
df.describe()
```

✓ 0.1s Python

rder_Processing_Time	Return_Rate	Customer_Rating	Warehouse_Capacity	Storage_Cost	Transportation_Cost	Backorder_Quantity	Damaged_Goods	Employee_Count
500.000000	500.000000	500.000000	500.000000	500.00000	500.000000	500.000000	500.000000	500.000000
2.750462	5.229366	3.090865	30032.818000	10263.83400	8001.086000	147.380000	24.684000	52.810000
1.262967	2.661448	1.156971	11921.277948	5584.14567	4269.354045	87.880656	14.868298	27.888087
0.505159	0.506755	1.004825	10338.000000	1083.00000	515.000000	0.000000	0.000000	5.000000
1.644623	3.085484	2.073853	19486.750000	5335.75000	4294.500000	71.750000	11.000000	28.000000
2.817771	5.128133	3.148014	29862.500000	10023.00000	7973.000000	141.000000	25.000000	53.000000
3.782273	7.406404	4.137274	40692.250000	15162.00000	11857.000000	224.000000	37.000000	78.000000
4.954139	9.992465	4.998258	49979.000000	19979.00000	14988.000000	300.000000	50.000000	100.000000

- **Feature Engineering →**

  - **Total Logistics Cost Column →** We combined operational, storage, and transportation costs into a single column. This gives a clear picture of the overall logistics expense instead of looking at separate cost components.
  - **Net Profit Column →** We subtracted total logistics cost from monthly sales to calculate net profit. This helps identify whether warehouses or suppliers are profitable after covering all logistics expenses.
  - **Operating Cost Ratio Column →** We divided total logistics cost by monthly sales to measure efficiency. A lower ratio means costs are well-managed compared to revenue, while a higher ratio signals inefficiency. This inefficiency can happen due to: Excessive Operational Costs (High labor), Storage Inefficiency (Paying for unused warehouse), and Transportation Issues (Expensive shipping routes).
  - **Delivery Efficiency Column →** We added shipping time, order processing time, and lead time to capture the total delivery cycle. This metric highlights how quickly products move from order to delivery.
  - **Stock Utilization Column →** We calculated the percentage of warehouse capacity being used by dividing current stock by total capacity. This shows whether storage space is underutilized or overloaded.
  - **Inventory Turnover Ratio Column →** We divided monthly sales by current stock to measure how fast inventory is sold. A higher ratio indicates efficient stock movement, while a lower ratio suggests slow-moving goods.
  - **Demand Gap Column →** We subtracted current stock from demand forecast to identify shortages or surpluses. **If the number is positive** → It means demand is higher than stock, so there's a **shortage**. **If the number is negative** → It means stock is more than demand, so there's a **surplus**.
  - **Risk Score Column →** We added damaged goods, return rate, and stockout risk into one score. This score shows how **risky or unreliable** the supply chain is: If the score is **high**, it means there are more problems — like too many damaged items, customers returning products, or frequent stockouts and if the score is **low**, it means the supply chain is running smoothly with fewer issues.
  - **New Features Grouping →** After creating several new metrics, we grouped them together into a single list. This makes it easier to focus only on the engineered KPIs instead of mixing them with the raw dataset fields.
  - **Missing Value Check →** We checked how many null values exist in each new metric.
  - **Summary Statistics →** We generated descriptive statistics (mean, median, min, max, quartiles) for the new metrics. This helps us understand the **distribution** of values.
  - **Final Dataset Export →** We saved the dataset into a CSV file for further analysis and visualization. This ensures the dataset is clean, complete, and ready for reporting or dashboard creation.

Total Logistics Cost : Total cost of all logistics operations -

```
df['Total_Logistics_Cost']=df['Operational_Cost']+df['Storage_Cost']+df['Transportation_Cost']
```

[24] ✓ 0.0s

Estimated Profit : Difference between monthly_sales and total_cost -

```
df['Net_Profit']=df['Monthly_Sales']-df['Total_Logistics_Cost']
```

[11] ✓ 0.0s

Calculate Operating Cost Ratio : This measures how much of sales revenue is consumed by logistics cost -

```
df['Operating_Cost_Ratio']=df['Total_Logistics_Cost']/df['Monthly_Sales']
```

[12] ✓ 0.0s

Delivery Efficiency : Total number of days taken for a customer to receive an order -

```
df['Delivery_Efficiency']=df['Shipping_Time_Days']+df['Order_Processing_Time']+df['Lead_Time_Days']
```

[13] ✓ 0.0s

Stock Utilization : Percentage of warehouse space being used-

```
df['Stock_Utilization']=(df['Current_Stock']/df['Warehouse_Capacity'])*100
```

14] ✓ 0.0s

Inventory Turnover Ratio : A way to measure how many times items are sold out and restocked-

```
df['Inventory_Turnover_Ratio'] = df['Monthly_Sales'] / df['Current_Stock']
```

15] ✓ 0.0s

Demand Gap : It shows how much more stock is needed-

```
df['Demand_Gap']=df['Demand_Forecast']-df['Current_Stock']
```

16] ✓ 0.0s

Risk Score : It gives overall supply chain risk-

```
df['Risk_Score']=df['Damaged_Goods']+df['Return_Rate']+df['Stockout_Risk']
```

```python
New_Cols=['Total_Logistics_Cost', 'Net_Profit', 'Operating_Cost_Ratio',
        'Delivery_Efficiency', 'Stock_Utilization', 'Inventory_Turnover_Ratio',
        'Demand_Gap', 'Risk_Score']
```

[19] ✓ 0.0s

```python
# Check the count of null values
df[New_Cols].isnull().sum()
```

[20] ✓ 0.0s

```
Total_Logistics_Cost        0
Net_Profit                  0
Operating_Cost_Ratio        0
Delivery_Efficiency         0
Stock_Utilization           0
Inventory_Turnover_Ratio    0
Demand_Gap                  0
Risk_Score                  0
dtype: int64
```

```python
# Check descriptive statistics
df[New_Cols].describe()
```

✓ 0.1s Python

	Total_Logistics_Cost	Net_Profit	Operating_Cost_Ratio	Delivery_Efficiency	Stock_Utilization	Inventory_Turnover_Ratio	Demand_Gap	Risk_Score
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	84151.274000	-78815.702000	22.653077	12.656462	11.097874	2.735464	234.228000	44.857366
std	21931.164833	22219.320918	17.934339	3.546652	8.208686	2.609801	2003.662878	15.799916
min	36523.000000	-130570.000000	4.558472	4.434040	1.091282	0.204481	-4275.000000	11.622787
25%	65828.500000	-97483.000000	10.636300	10.183141	5.266148	1.141594	-1188.500000	31.607661
50%	84625.000000	-78697.000000	15.811373	12.555027	9.128269	1.942821	209.000000	45.162090
75%	102163.250000	-61388.750000	27.866772	15.219212	14.131303	3.148789	1747.250000	57.339818
max	132412.000000	-28877.000000	102.487052	21.707222	44.248372	17.527574	4575.000000	78.202471

```python
df.to_csv('Final_Supply_Chain_Analysis.csv',index=False)
```

✓ 0.0s Python

Generate + Code + Markdown

6. Data Analysis using SQL (Supply Chain Operations)

We performed structured analysis in PostgreSQL to answer key business questions:

1. **Profit & Operating Cost Analysis →** Identified cities with negative profit (Loss) and checked if high logistics costs are affecting their overall cost efficiency ratio.

	location character varying (50)	avg_net_profit numeric	total_cost numeric	avg_cost_ratio numeric
1	Chicago	-74004.86	3517743.00	17.95
2	Dallas	-74708.61	4077963.00	21.64
3	Atlanta	-76869.93	4552766.00	19.06
4	New York	-78265.37	4513770.00	23.72
5	Houston	-78413.70	3698939.00	21.90
6	Denver	-78840.76	3882388.00	19.82
7	San Francisco	-79588.04	4417654.00	22.80
8	Seattle	-80860.07	3748706.00	28.87
9	Miami	-81468.84	4859521.00	21.72
10	Los Angeles	-84292.00	4806187.00	28.82

Total rows: 10 Query complete 00:00:00.128

2. **Impact on Customers Rating due to Delivery Delay (Slow deliveries = Unhappy customers) →** Evaluated the impact of low 'Delivery Efficiency' (more than 6 days) on Customer Ratings.

	location character varying (50)	avg_wait_time numeric	avg_customer_rating numeric
1	Los Angeles	12.88	2.81
2	New York	13.25	2.95
3	Denver	13.03	2.97
4	Atlanta	12.50	3.03
5	San Francisco	12.17	3.05
6	Houston	11.71	3.11
7	Seattle	11.10	3.15
8	Chicago	13.31	3.19
9	Dallas	13.37	3.29
10	Miami	12.97	3.36

Total rows: 10 Query complete 00:00:00.135

3. **Warehouse Space & Utilization by Product Category** → Analyzed warehouse capacity utilization by product category to identify underutilized storage space.

Data Output | Messages | Notifications

	product_category character varying (50)	avg_stock_utilization numeric
1	Electronics	11.74
2	Furniture	11.63
3	Automotive	11.01
4	Groceries	10.69
5	Apparel	10.54

4. **Product-wise Sales Performance** → Analyzed product-wise sales performance to identify top and low-performing categories.

	product_category character varying (50)	total_monthly_sales bigint	avg_monthly_sales numeric
1	Apparel	564991	4999.92
2	Electronics	560057	5600.57
3	Groceries	541821	5160.20
4	Furniture	516458	5553.31
5	Automotive	484459	5443.36

5. **Demand vs Stock Comparison** → Compared current stock against demand forecast to identify shortages or surpluses.

	product_category character varying (50)	total_stock bigint	total_demand bigint	total_demand_gap bigint
1	Electronics	275000	310806	35806
2	Automotive	234425	261340	26915
3	Apparel	301653	320878	19225
4	Furniture	253736	271495	17759
5	Groceries	296597	314006	17409

6. **Top Delayed Warehouses** → Identified warehouses with the lowest Delivery Efficiency (Shipping + Processing).

	warehouse_id character varying (20)	location character varying (50)	delivery_efficiency numeric (10,2)	customer_rating numeric (10,2)
1	WH414	Chicago	21.71	2.14
2	WH298	New York	21.43	3.95
3	WH473	Atlanta	21.38	4.28
4	WH492	Dallas	20.83	4.70
5	WH023	San Francisco	20.62	2.02
6	WH352	Los Angeles	20.42	1.77
7	WH092	Miami	20.36	2.67
8	WH131	Denver	20.29	1.19
9	WH042	Chicago	20.27	1.01
10	WH444	New York	20.13	2.08
11	WH062	Chicago	19.77	4.82
12	WH434	New York	19.76	2.90
13	WH490	Los Angeles	19.60	3.19
14	WH063	Denver	19.56	4.35

Total rows: 492 Query complete 00:00:00.167

7. **Top 10 Warehouses by Order Processing Time** → Identified top 10 warehouses with the longest order processing times for performance review.

	warehouse_id character varying (20)	order_processing_time numeric (10,2)
1	WH286	4.95
2	WH113	4.95
3	WH381	4.94
4	WH192	4.92
5	WH303	4.91
6	WH387	4.91
7	WH349	4.90
8	WH286	4.90
9	WH156	4.89
10	WH204	4.85

8. **Stockout Risk by Location** → Summarized average, maximum, and minimum stockout risk across cities.

Showing rows: 1 to 10 Page No:

	location character varying (50)	avg_stockout_risk numeric	max_stockout_risk integer	min_stockout_risk integer	warehouse_count bigint
1	Houston	16.2727272727272727	25	5	44
2	New York	16.0000000000000000	25	5	54
3	Dallas	15.9411764705882353	25	6	51
4	Chicago	15.1136363636363636	25	6	44
5	Seattle	15.0681818181818182	25	6	44
6	Atlanta	14.8000000000000000	25	5	55
7	San Francisco	14.5000000000000000	25	5	52
8	Los Angeles	14.1851851851851852	25	5	54
9	Miami	14.0357142857142857	25	5	56
10	Denver	13.7173913043478261	24	5	46

9. **Risk by Product Category →** Summarized average, maximum, and minimum risk scores for each product category to identify which categories carry the highest operational risk.

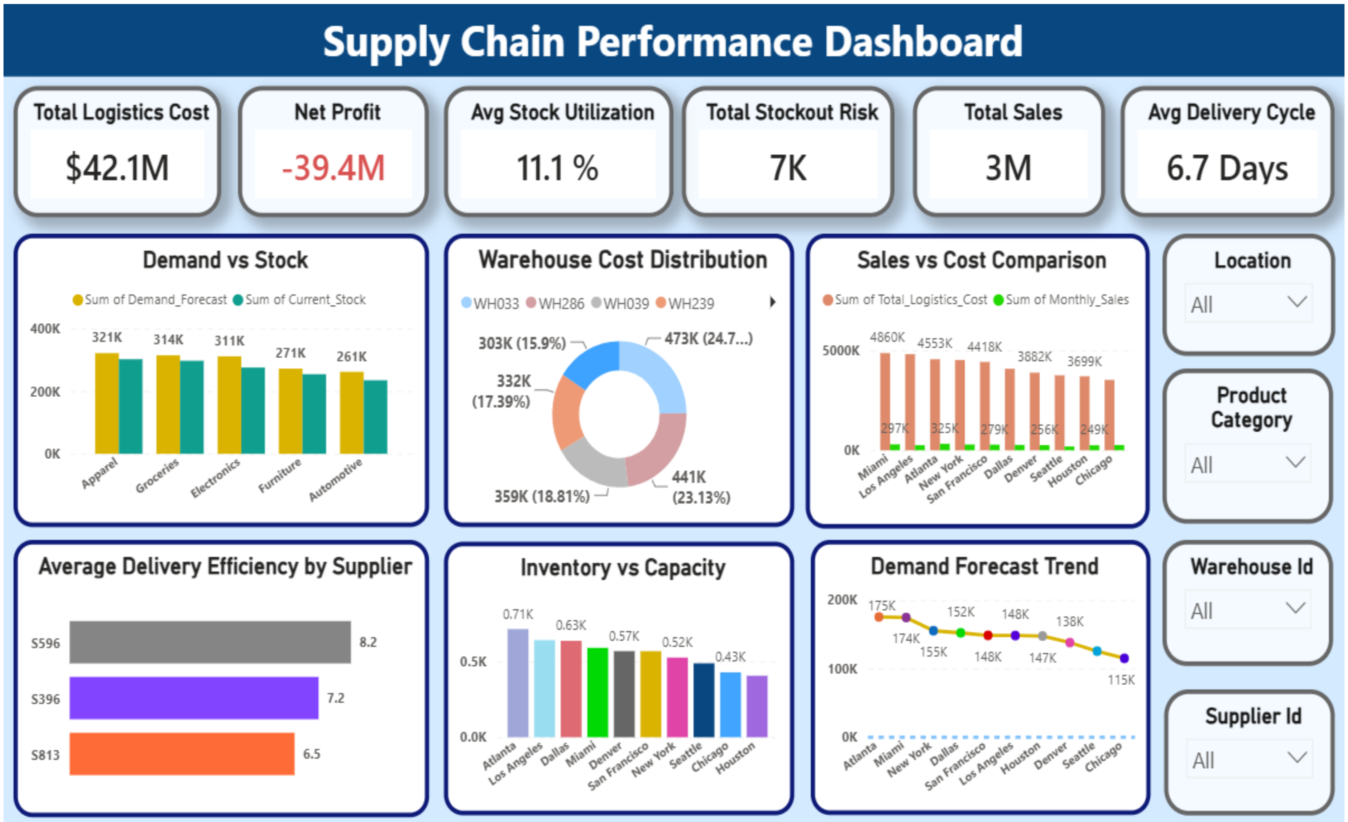
	product_category character varying (50)	avg_risk numeric	min_risk numeric	max_risk numeric	total_warehouses bigint
1	Electronics	47.13	11.62	76.09	100
2	Apparel	45.85	15.76	75.09	113
3	Automotive	44.45	14.53	75.53	89
4	Groceries	44.41	12.22	78.20	105
5	Furniture	42.10	12.86	76.89	93

10. **Supplier Performance →** Evaluated 10 worst suppliers based on their overall contribution to operations. (This helps management identify suppliers who are profitable vs who contribute losses.)

	supplier_id character varying (20)	warehouse_count bigint	avg_stockout_risk numeric	avg_delivery_effieciency numeric	avg_profit numeric
1	S142	1	9.00	13.11	-126708.00
2	S268	1	13.00	7.00	-125045.00
3	S533	1	6.00	10.97	-121394.00
4	S728	1	15.00	15.68	-121354.00
5	S183	1	19.00	15.67	-120561.00
6	S312	1	16.00	16.31	-120262.00
7	S928	1	25.00	13.84	-119328.00
8	S261	1	25.00	17.74	-119281.00
9	S504	1	12.00	8.98	-118859.00
10	S588	1	13.00	9.11	-115414.00

7. Dashboard in Power BI

Finally, we built an interactive dashboard in Power BI to present insights visually.



8. Business Recommendations

- **Cut Costs →** Try to spend less on transport, storage, and daily operations so profit goes up.

- **Deliver Faster →** Reduce waiting time by speeding up order handling and shipping. Customers will be happier.

- **Use Warehouse Space Better →** Don’t keep warehouses too empty or too full. Balance stock so space is used wisely.

- **Sell Stock Quickly →** Move products faster with offers, bundles, or promotions. This avoids goods sitting too long.

- **Match Demand and Supply →** Keep enough stock to meet customer demand but avoid overstocking. Plan purchases carefully.

- **Lower Risks →** Reduce damaged goods, returns, and stockouts by improving packaging, quality checks, and backup stock.

- **Focus on Profitable Products →** Spend more effort on categories that sell well and give good profit, like electronics or apparel.

- **Check Discounts and Returns →** Make sure discounts increase sales without cutting too much profit. Also, reduce high return rates by improving product quality.

- **Train Employees →** Teach staff better ways to handle stock and orders. This saves time and reduces mistakes.